

# Função lambda

Nas práticas de programação, eficiência e clareza são essenciais, principalmente ao manipular grandes volumes de dados. Segundo Guilhon *et al.* (2022), Python se destaca nesse contexto por oferecer recursos que tornam o código mais enxuto e expressivo, favorecendo a legibilidade e a manutenção. Um desses recursos é a função lambda, que permite simplificar operações curtas sem a necessidade de criar funções nomeadas tradicionais. Como aponta Lambert (2022), compreender a sintaxe e a aplicação dessas funções é fundamental para aproveitar seu potencial na manipulação de dados, especialmente quando combinadas com ferramentas como `sort`, `map` e `list`, que ampliam sua eficiência e versatilidade.

Neste capítulo, exploraremos o conceito e as aplicações das funções lambda, mostrando como elas podem tornar o código mais compacto e legível. Na primeira seção, abordaremos a sintaxe e alguns usos práticos. Em seguida, veremos como combiná-las com funções como `sort`, `map` e `list`, ampliando o potencial de manipulação eficiente de dados.

# 1 Conceito e aplicações

Desenvolver aplicações e exercer o ato de programar consistem em prática constante de exercícios e métodos que resultam em resolver problemas de maneira eficiente e no menor tempo possível. Quando desenvolvemos aplicações em Python, organizamos nossos códigos em funções para um melhor aproveitamento, assim como aplicamos ações de refatoração. No entanto, podemos nos deparar com situações em que funções resolvem expressões curtas e simples, principalmente associadas a funções aritméticas ou de fatiamento de dados. Para essas situações, o Python possibilita o desenvolvimento de subprogramas em única linha, intitulada função lambda.

A função lambda recebe esse nome devido ao lambda calculus (cálculo lambda), um sistema matemático formal desenvolvido por Alonzo Church na década de 1930. Esse cálculo é um modelo de computação baseado em funções anônimas e expressões matemáticas puras, sem a necessidade de atribuições ou estados mutáveis.

Para escrever uma função lambda em Python, utiliza-se seguinte estrutura:

```
lambda parâmetros: expressão
```

Na qual *expressão* deve ter uma única linha e gerar o valor de retorno, e os *parâmetros*, separados entre vírgulas, para entradas de dados na expressão. Por exemplo: podemos ter uma função tradicional escrita conforme o exemplo 1:

## Exemplo 1

```
def soma(x, y):  
    return x + y  
print(soma(3, 4)) # Saída: 7
```

Nesse caso, podemos montar uma função lambda equivalente da seguinte maneira:

```
soma = lambda x, y: x + y  
print(soma(3, 4)) # Saída: 7
```

É importante ressaltar que funções definidas com `def` podem ter múltiplas linhas e comentários e são mais legíveis. Em contrapartida, funções lambda são úteis para funções simples e descartáveis, usadas dentro de outras expressões.

Em situações em que a função é curta e utilizada apenas temporariamente, a aplicação de funções anônimas, como as funções lambda, pode tornar o código mais enxuto e direto, eliminando a necessidade de definições formais que seriam usadas apenas uma vez. Isso é especialmente útil quando se deseja evitar a criação de funções nomeadas para tarefas simples, como pequenas transformações de dados ou operações pontuais, garantindo uma abordagem mais concisa e legível no desenvolvimento.

Em Python, funções lambda não apresentam vantagens significativas de desempenho em relação às funções tradicionais definidas com `def`, pois ambas são tratadas internamente como objetos de primeira classe e executadas pelo interpretador da mesma forma. No entanto, lambda pode ter uma leve sobrecarga devido à sua limitação a uma única expressão e à impossibilidade de conter múltiplas instruções, tornando-as menos legíveis e mais difíceis de depurar. Além disso, ao serem usadas excessivamente em contextos complexos, como loops aninhados ou operações de alta carga, podem comprometer a clareza do código sem ganhos de eficiência, já que o Python não realiza otimizações específicas para lambda, diferentemente de linguagens compiladas que aplicam técnicas como inlining.



## PARA SABER MAIS

As funções lambda em Python são úteis para expressões curtas e rápidas, mas sua utilização excessiva pode dificultar a depuração e manutenção do código. Para evitar erros comuns, como escopo inadequado de variáveis e uso indevido em expressões complexas, é recomendável testá-las em pequenos trechos de código antes de integrá-las a um programa maior.

Na área de probabilidade e estatística, Python oferece bibliotecas poderosas como NumPy, SciPy, pandas e statsmodels, que permitem cálculos estatísticos, modelagem de distribuições, testes de hipóteses e visualização de dados. Funções lambda são frequentemente usadas em conjunto com `map()`, `filter()` e `apply()` para manipulação eficiente de dados estatísticos, mas sua aplicação deve ser equilibrada com funções normais (`def`) para garantir clareza e reusabilidade do código.

Se deseja aprofundar seus conhecimentos sobre a correta utilização de funções lambda e programação estatística em Python, explore conceitos como funções de distribuição de probabilidade, regressão estatística e processamento vetorial com NumPy.

---

## 2 Uso das funções sort, map e list

A linguagem de programação Python oferece diversas funções embutidas para manipulação de listas e coleções de dados, sendo `sort`, `map` e `list` algumas das mais utilizadas. Cada uma delas tem uma aplicação específica e pode ser combinada com funções lambda para tornar o código mais conciso e eficiente.

Por exemplo, a função `sort()` é utilizada para ordenar listas in-place (ou seja, altera a lista original). Já `sorted()` retorna uma nova lista ordenada sem modificar a original. Observe, no exemplo 2, o código para analisar as diferenças. Lembrando que o argumento `reverse=True` permite realizar a ordenação de forma decrescente.

## Exemplo 2

```
numeros = [5, 2, 9, 1, 5, 6]

# Ordenando a própria lista
numeros.sort()
print(numeros) # [1, 2, 5, 5, 6, 9]

# Ordenando sem modificar a lista original
nova_lista = sorted(numeros, reverse=True)
print(nova_lista) # [9, 6, 5, 5, 2, 1]
```

Podemos utilizar a opção `key` para definir critérios específicos, como ordenar uma lista de tuplas pelo segundo elemento, associando o recurso de função `lambda` para selecionar o elemento desejado da estrutura.

```
pessoas = [("Alice", 25), ("Bob", 30), ("Carol", 20)]
pessoas.sort(key=lambda x: x[1]) # Ordena pela idade
print(pessoas) # [('Carol', 20), ('Alice', 25), ('Bob', 30)]
```

Por sua vez, a função `map()` aplica uma função a cada item de um iterável, como listas e tuplas, retornando um iterador. Para obter uma lista, é necessário converter o resultado com `list()`, como o exemplo 3, que dobra os valores de uma lista.

## Exemplo 3

```
numeros = [1, 2, 3, 4, 5]
dobrados = list(map(lambda x: x * 2, numeros))
print(dobrados) # [2, 4, 6, 8, 10]
```

Em um outro exemplo, o `map()` pode ser usado para formatar uma lista de strings, como apresentado a seguir, no exemplo 4.

## Exemplo 4

```
nomes = ["alice", "bob", "carol"]
nomes_maiusculos = list(map(str.upper, nomes))
print(nomes_maiusculos) # ['ALICE', 'BOB', 'CAROL']
```

Por último, vamos fazer referência especificamente à função `list()`, usada para converter iteráveis (Iterator) em listas. Isso é útil ao trabalhar com iteradores, `map()`, `filter()`, `zip()`, entre outros.

O exemplo 5 é a conversão da saída da função `range()` para uma lista.

#### Exemplo 5

```
numeros = list(range(5))  
print(numeros) # [0, 1, 2, 3, 4]
```

Por sua vez, como já apresentado anteriormente, a função `list()` também converte iteradores em listas geradas pela função `map()`, como apresenta o exemplo 6 a seguir.

#### Exemplo 6

```
valores = map(lambda x: x**2, [1, 2, 3, 4])  
valores_lista = list(valores)  
print(valores_lista) # [1, 4, 9, 16]
```

Conhecendo as simplificações das funções `lambda` e de iteradores como o `map`, pode-se resolver problemas com mais facilidade e menos linhas de código. Vamos ao exemplo 7.

#### Exemplo 7

*Suponha que você recebeu uma lista de temperaturas em graus Celsius e precisa convertê-las para Fahrenheit usando Python.*

Primeiramente, vamos resolver este problema utilizando um código imperativo e tradicional, sem o uso de funções.

```
# Lista de temperaturas em Celsius  
celsius = [0, 10, 20, 30, 40, 100]  
  
# Criando uma lista vazia para armazenar os resultados  
fahrenheit = []
```

```
# Iterando sobre a lista e convertendo cada valor
for temp in celsius:
    f = (temp * 9/5) + 32 # Aplicando a fórmula de conversão
    fahrenheit.append(f) # Adicionando o resultado à nova
    lista

# Exibindo a lista de temperaturas em Fahrenheit
print(fahrenheit)
```

Criamos uma lista vazia chamada *fahrenheit* para armazenar os valores convertidos e, em seguida, percorremos a lista *celsius* usando um loop *for*, processando cada temperatura individualmente. Durante a iteração, aplicamos a fórmula de conversão:

$$F = \left(C \times \frac{9}{5}\right) + 32$$

Depois, armazenamos o resultado na variável *f*. Esse valor convertido é então adicionado à lista *fahrenheit* utilizando o método *.append()*. Após a conclusão do loop, imprimimos a lista resultante contendo todas as temperaturas convertidas de Celsius para Fahrenheit.

Contudo, vejamos como podemos refatorar esse código para o uso de funções tradicionais, como mostra o exemplo 8.

### Exemplo 8

```
# Função para converter Celsius para Fahrenheit
def celsius_para_fahrenheit(c):
    return (c * 9/5) + 32

# Lista de temperaturas em Celsius
celsius = [0, 10, 20, 30, 40, 100]

# Criando uma lista vazia para armazenar as temperaturas
# convertidas
fahrenheit = []

# Percorrendo a lista celsius elemento por elemento
```

```

for temp in celsius:
    # Convertendo a temperatura atual de Celsius para
    Fahrenheit
    temp_fahrenheit = celsius_para_fahrenheit(temp)

    # Adicionando a temperatura convertida à lista fahrenheit
    fahrenheit.append(temp_fahrenheit)

# Exibindo a lista final de temperaturas convertidas
print(fahrenheit)

```

Nesse caso, a fórmula para a transformação de escala de temperatura foi encapsulada na função `celsius_para_fahrenheit()`, seguindo uma melhor organização e encapsulamento. No entanto, podemos otimizar ainda mais essa solução utilizando funções `map()` e `lambda()`, conforme exemplo 9.

### Exemplo 9

```

# Lista de temperaturas em Celsius
celsius = [0, 10, 20, 30, 40, 100]

# Aplicando a conversão com map() e lambda
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsius))

# Exibindo o resultado
print(fahrenheit)

```

Criamos uma lista contendo temperaturas em Celsius e utilizamos a função `map()`, que permite aplicar uma transformação a cada elemento dessa lista de forma eficiente. Dentro do `map()`, empregamos uma função `lambda` para aplicar diretamente a fórmula de conversão de Celsius para Fahrenheit, tornando o código mais conciso. Como `map()` retorna um iterador, utilizamos `list()` para convertê-lo em uma lista, garantindo que os valores possam ser manipulados e exibidos facilmente. Por fim, imprimimos a nova lista contendo as temperaturas já convertidas para Fahrenheit.

## Considerações finais

As funções lambda são um recurso poderoso em Python para simplificar operações curtas e tornar o código mais enxuto e expressivo, especialmente quando usadas em conjunto com funções como `sort`, `map` e `list`. Embora sua utilização facilite a escrita de código conciso, é essencial equilibrar seu uso com funções tradicionais (`def`) para garantir legibilidade e manutenção. Em contextos mais complexos, o uso excessivo de funções lambda pode comprometer a clareza do código, tornando a depuração mais difícil. Portanto, compreender quando e como aplicá-las corretamente é fundamental para um desenvolvimento eficiente e organizado em Python.

## Referências

GUILHON, André *et al.* (org.). **Jornada Python**: uma jornada imersiva na aplicabilidade de uma das mais poderosas linguagens de programação do mundo. Rio de Janeiro, RJ: Brasport, 2022. *E-book*.

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. Cengage Learning Brasil, 2022. *E-book*.

